Beyond Vibe Coding

Building Context-Aware Engineering Workflows

Who Am I?

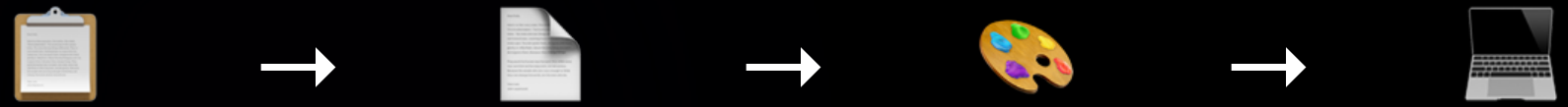
Asad Ullah Khalid

- 🚗 Senior Frontend Developer
- ⏱️ Joined MB.io in Aug 2023
- 🔄 ART: MyMB (MMV)
- 💻 Fullstack JS/TS
- 📍 Lives in Berlin & From Pakistan

Live feedback

ANONYMOUS

How every task starts



Read ticket → Check specs → Review designs → Explore code

Write code → Write tests → Create PR

Most of the time isn't coding. It's context-engineering.

The overhead no one talks about

Context switching

Between 5+ tools before writing a line

Mental overhead

Re-building context every single time

Repetitive setup

Before any real engineering begins

What if the context-engineering just happened automatically?

So I built something

An AI workflow that gathers the information and engineer the context.

Input

PROMPT

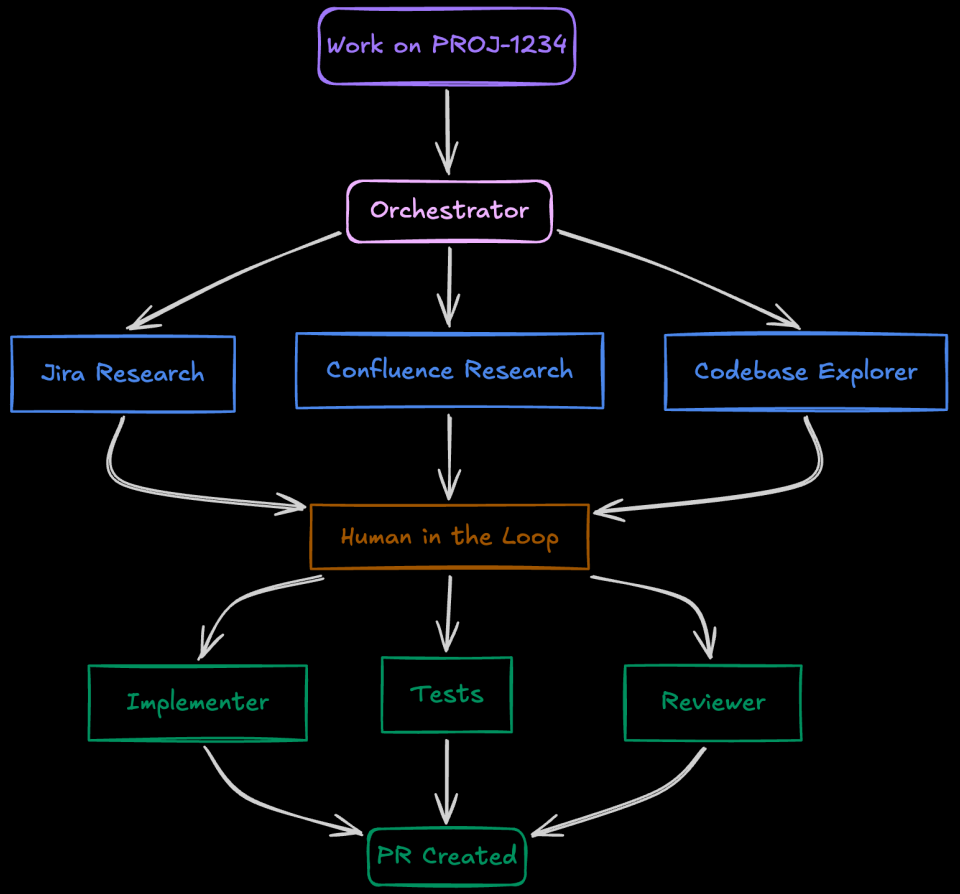
Work on USGMMV-1234

Output

RESULT

Context gathered. Code written. PR created.

The system behind it



What it actually looks like

WHAT THE SYSTEM HANDLES	WHAT STAYS WITH YOU
Fetches the ticket	Reviewing the context summary
Pulls relevant Confluence docs	Answering edge case questions
Explores the codebase	Guiding implementation decisions
Writes and reviews code	Approving before merge
Opens the PR	**Engineering judgment**

Four building blocks

Agents

who does the work

Prompts

how you trigger it

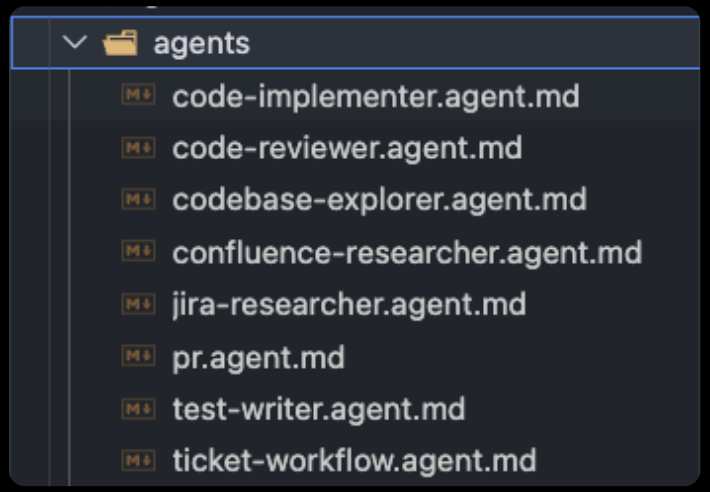
Skills

what they know

MCPs

where they get data

One responsibility per agent



Started with one big agent that did everything

↓

Unreliable. Hard to debug. Unpredictable.

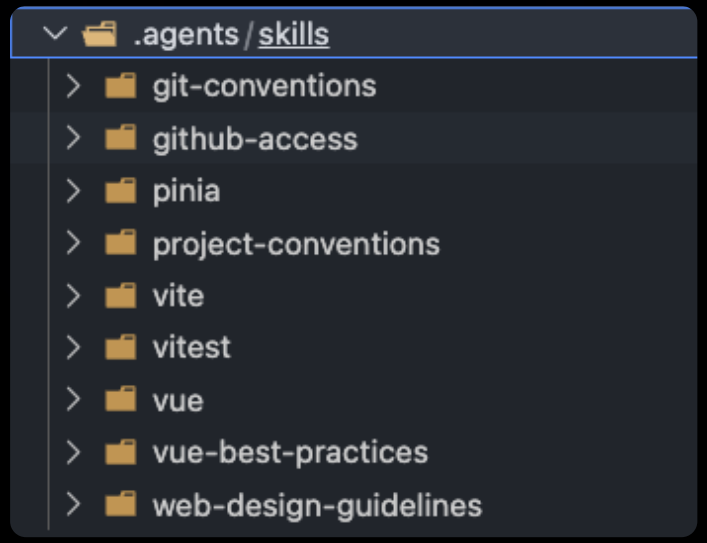
↓

Split into specialists

↓

Predictable. Debuggable. Each usable standalone.

Skills — shared knowledge



project-conventions
← loaded by implementer, test-writer, reviewer

git-conventions
← loaded by pr-creator

vue / pinia / vitest
← loaded on demand

MCPs — the bridge to your tools

AI ↔ Jira

AI ↔ Confluence

AI ↔ GitHub Enterprise

One-time JSON config.


```
"servers": {
  ▷Start | 43 tools | 2 prompts | More...
  "github-ghe": {
    "type": "http",
    "url": "https://copilot-api.mercedes-benz.ghe.com/mcp",
    "headers": {
      "Authorization": "Bearer ${env:MB_GITHUB_GHE_PAT}"
    }
  },
  ▷Start | 73 tools | More...
  "mcp-atlassian": {
    "command": "uvx",
    "args": ["mcp-atlassian"],
    "env": {
      "JIRA_URL": "https://jira.mercedes-benz.io",
      "JIRA_PERSONAL_TOKEN": "${env:MB_JIRA_PAT}",
      "CONFLUENCE_URL": "https://confluence.mercedes-benz.io",
      "CONFLUENCE_PERSONAL_TOKEN": "${env:MB_CONFLUENCE_TOKEN}"
    }
  }
}
```

Start with one problem

Hate writing tests?

Start with a test-writer agent

PRs always incomplete?

Start with a PR creator agent

Legacy code a maze?

Start with a codebase explorer

How to start today

1. Create `.agent.md` — **one clear responsibility**
2. Add one **MCP connection** — Jira, GitHub, whatever you use daily
3. Write a **prompt** — the trigger that kicks it off
4. **Iterate** — it won't be perfect. That's expected.

To recap

Context-gathering is automatable — most pre-coding work is just data retrieval

The architecture is simple — agents, prompts, skills, MCPs — all text files

You don't need the full pipeline — start with your biggest friction point

The interesting work stays with you — decisions, architecture, judgment

Questions?

SCAN FOR FEEDBACK & LINKS

asadullahkhalid.com